

Index / Table of Contents

Exp. No.	Title	Page
1	Understanding DevOps Principles, Practices, Roles and Responsibilities	_____
2	Installation and Configuration of Git and GitHub	_____
3	Perform Git Operations on Local and Remote Repositories	_____
4	Installation and Configuration of Jenkins with Maven to Setup a Build Job	_____
5	Build Jenkins Pipeline and Deploy Application on Apache Tomcat Server	_____
6	Configure Jenkins Master-Agent Architecture	_____
7	Setup and Run Selenium Tests in Jenkins Using Maven	_____
8	Docker Installation and Container Management	_____
9	Create Dockerfile and Build Docker Image for a Sample Web Application	_____

Experiment No. 1

Understanding DevOps Principles, Practices, Roles and Responsibilities

Aim

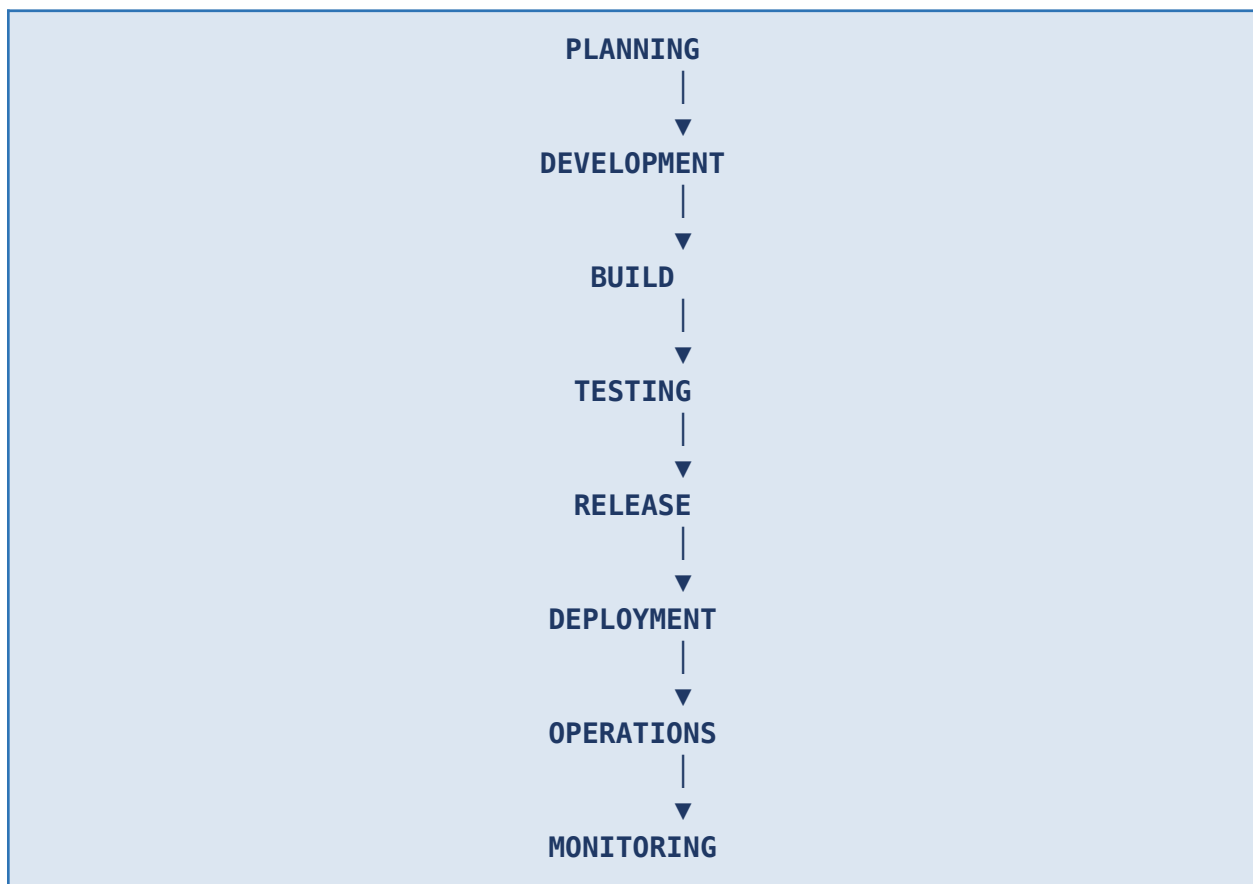
To understand the fundamentals of DevOps, its principles, lifecycle, practices, and the roles and responsibilities of a DevOps Engineer.

Required Software

- Windows 10/11
- Web Browser (Chrome/Edge)
- Internet Connection
- Visual Studio Code or Notepad

Program / Content

DevOps Lifecycle



DevOps Tools

DevOps Phase	Tools
Planning	Jira, Trello
Version Control	Git, GitHub
Continuous Integration	Jenkins
Build Tool	Maven, Gradle, Ant
Testing	Selenium, JUnit
Containerization	Docker
Orchestration	Kubernetes
Configuration Management	Puppet, Ansible
Monitoring	Prometheus, Grafana

Roles and Responsibilities of a DevOps Engineer

- Collaborate with Development and Operations teams.
- Automate software build and deployment processes.
- Configure Continuous Integration and Continuous Delivery (CI/CD).
- Monitor application performance and system health.
- Manage source code using Git and GitHub.
- Maintain Docker containers and Kubernetes clusters.
- Ensure application security and infrastructure reliability.
- Troubleshoot deployment and production issues.

DevOps Principles

- Collaboration
- Automation
- Continuous Integration (CI)
- Continuous Delivery (CD)
- Continuous Monitoring
- Infrastructure as Code (IaC)
- Continuous Feedback
- Security Integration (DevSecOps)

Algorithm

1. Study the concept of DevOps and its importance in software development.

2. Understand the DevOps lifecycle from Planning to Monitoring.
3. Identify the tools used in each phase of the DevOps lifecycle.
4. Learn the principles of Continuous Integration and Continuous Delivery.
5. Study the roles and responsibilities of a DevOps Engineer.
6. Analyze how automation improves software development and deployment.
7. Observe the relationship between Development and Operations teams.
8. Record the observations and summarize the learning outcomes.

Result

Thus, the DevOps principles, lifecycle, practices, tools, and roles of a DevOps Engineer were studied successfully.

Output



SCREENSHOT PLACEHOLDER

Paste Screenshot of DevOps Lifecycle Diagram or DevOps Tools Chart Here

Experiment No. 2

Installation and Configuration of Git and GitHub

Aim

To install Git, create a GitHub account, configure Git with user information, and verify the installation.

Required Software

- Windows 10/11
- Git
- GitHub Account
- Command Prompt or Git Bash
- Web Browser
- Internet Connection

Program / Commands

Check Git Installation

```
git --version
```

Configure Git Username

```
git config --global user.name "Your Name"
```

Configure Git Email

```
git config --global user.email "youremail@example.com"
```

Verify Git Configuration

```
git config --list
```

Display Configured Username

```
git config user.name
```

Display Configured Email

```
git config user.email
```

Algorithm

1. Download the latest version of Git from the official Git website.
2. Install Git using the default installation settings.
3. Open Command Prompt or Git Bash after the installation is complete.
4. Verify the Git installation using the `git --version` command.
5. Configure the Git username using the `git config --global user.name` command.
6. Configure the Git email address using the `git config --global user.email` command.
7. Verify the configured username and email using the `git config --list` command.
8. Create a GitHub account and sign in successfully.

Result

Thus, Git was installed successfully, configured with the user information, and connected for use with GitHub.

Output

 SCREENSHOT PLACEHOLDER

Paste Screenshot of `git --version` and `git config --list` Output Here

Experiment No. 3

Perform Git Operations on Local and Remote Repositories

Aim

To perform various Git operations on local and remote repositories using Git commands.

Required Software

- Windows 10/11
- Git
- GitHub Account
- Command Prompt or Git Bash
- Visual Studio Code (Optional)
- Internet Connection

Program / Commands

Create a New Project Folder

```
mkdir DevOpsLab  
cd DevOpsLab
```

Initialize a Local Git Repository

```
git init
```

Check Repository Status

```
git status
```

Create a Sample File

```
echo Hello DevOps > index.txt
```

Add File to Staging Area

```
git add .
```

Commit the Changes

```
git commit -m "Initial Commit"
```

Create a New Branch

```
git branch feature
```

Switch to the New Branch

```
git checkout feature
```

Display Available Branches

```
git branch
```

Connect Local Repository to GitHub

```
git remote add origin https://github.com/username/DevOpsLab.git
```

Push Project to GitHub

```
git branch -M main  
git push -u origin main
```

Pull Latest Changes

```
git pull origin main
```

Clone a Remote Repository

```
git clone https://github.com/username/DevOpsLab.git
```

View Commit History

```
git log
```

Algorithm

1. Create a new project folder and initialize it as a local Git repository.
2. Create a sample file and check the repository status.
3. Add the file to the staging area using the git add command.
4. Commit the staged changes with an appropriate commit message.
5. Create a new repository in GitHub and copy its repository URL.
6. Connect the local repository to the remote GitHub repository.
7. Push the committed files to the remote repository and verify the upload.
8. Perform additional Git operations such as branch creation, pull, clone, and log to manage the repository.

Result

Thus, various Git operations on local and remote repositories were performed successfully using Git commands.

Output



SCREENSHOT PLACEHOLDER

Paste Screenshots of git status, git commit, git push, and GitHub Repository Here

Experiment No. 4

Installation and Configuration of Jenkins with Maven to Setup a Build Job

Aim

To install and configure Jenkins with Maven and create a Freestyle Build Job for automating the software build process.

Required Software

- Windows 10/11
- Java JDK 21 or later
- Jenkins (LTS Version)
- Apache Maven
- Git
- GitHub Account
- Web Browser (Chrome/Edge)
- Internet Connection

Program / Commands

Verify Java Installation

```
java -version
```

Verify Git Installation

```
git --version
```

Verify Maven Installation

```
mvn -version
```

Open Jenkins Dashboard

```
http://localhost:8080
```

Maven Build Command

```
clean install
```

Optional Maven Commands

```
clean  
compile  
test  
package  
install
```

Algorithm

1. Install Java JDK and verify the installation using the `java -version` command.
2. Install Jenkins and start the Jenkins service.
3. Open the Jenkins dashboard using `http://localhost:8080` and complete the initial setup.
4. Install Apache Maven and verify the installation using the `mvn -version` command.
5. Configure Maven in Manage Jenkins → Global Tool Configuration.
6. Create a new Freestyle Project in Jenkins.
7. Configure the source code repository and specify the Maven build goal (clean install).
8. Save the project and click Build Now to execute the build successfully.

Result

Thus, Jenkins was successfully installed, Maven was configured, and a Freestyle Build Job was executed successfully.

Output



SCREENSHOT PLACEHOLDER(S)

- *Java Version (java -version)*
- *Maven Version (mvn -version)*
 - *Jenkins Dashboard*
- *Global Tool Configuration (Maven)*
 - *Freestyle Project Configuration*
 - *Build Now*
- *Console Output showing BUILD SUCCESS*

Experiment No. 5

Build Jenkins Pipeline and Deploy Application on Apache Tomcat Server

Aim

To create a Jenkins Pipeline using Maven and deploy a web application on the Apache Tomcat server.

Required Software

- Windows 10/11
- Java JDK 21 or later
- Jenkins
- Apache Maven
- Apache Tomcat
- Git
- GitHub Account
- Web Browser

Program / Commands

Jenkins Pipeline (Jenkinsfile)

```
pipeline {
  agent any

  stages {

    stage('Checkout') {
      steps {
        git 'https://github.com/username/project.git'
      }
    }

    stage('Build') {
      steps {
        bat 'mvn clean package'
      }
    }

    stage('Deploy') {
      steps {
        bat 'copy target\\*.war "C:\\Program Files\\Apache Software
Foundation\\Tomcat 10.1\\webapps\\"'
```

```

    }
  }
}

```

Maven Build

```
mvn clean package
```

Algorithm

1. Install and configure Apache Tomcat Server.
2. Create a Pipeline Project in Jenkins.
3. Connect the GitHub repository to Jenkins.
4. Create the Jenkins Pipeline script (Jenkinsfile).
5. Build the project using the Maven command clean package.
6. Deploy the generated WAR file to the Tomcat webapps directory.
7. Start or restart the Tomcat server.
8. Open the deployed application in the browser and verify successful deployment.

Result

Thus, the Jenkins Pipeline was created successfully, and the web application was built and deployed on the Apache Tomcat server.

Output



SCREENSHOT PLACEHOLDER(S)

- Jenkins Pipeline Project
 - Jenkinsfile
 - Build Console Output
 - BUILD SUCCESS
- Tomcat Web Application Running

Experiment No. 6

Configure Jenkins Master-Agent Architecture

Aim

To configure Jenkins Master-Agent architecture for distributed build execution.

Required Software

- Windows 10/11 or Linux
- Java JDK
- Jenkins
- Network Connection
- Web Browser

Program / Commands

Download Jenkins Agent

```
java -jar agent.jar ^  
-jnlpUrl http://localhost:8080/computer/Agent/slave-agent.jnlp ^  
-secret YOUR_SECRET ^  
-workDir "C:\JenkinsAgent"
```

Verify Java

```
java -version
```

Algorithm

1. Install Jenkins on the Master system.
2. Create a new Agent node in Jenkins.
3. Install Java on the Agent machine.
4. Download the Jenkins Agent (agent.jar).
5. Connect the Agent to the Master using the JNLP command.
6. Verify that the Agent status changes to Online.
7. Assign a build job to the Agent.
8. Execute the build and verify successful completion.

Result

Thus, Jenkins Master-Agent architecture was configured successfully, and distributed build execution was performed.

Output



SCREENSHOT PLACEHOLDER(S)

- *Jenkins Nodes Page*
- *Agent Configuration*
- *Agent Online Status*
- *Build Executed on Agent*

Experiment No. 7

Setup and Run Selenium Tests in Jenkins Using Maven

Aim

To execute Selenium automation test cases using Jenkins and Maven.

Required Software

- Windows 10/11
- Java JDK
- Selenium WebDriver
- Apache Maven
- Jenkins
- Google Chrome
- ChromeDriver
- Git

Program / Commands

Execute Selenium Test

```
mvn test
```

Jenkins Build Goal

```
clean test
```

Sample TestNG XML

```
<suite name="TestSuite">
  <test name="ChromeTest">
    <classes>
      <class name="Test.LoginTest"/>
    </classes>
  </test>
</suite>
```

Algorithm

1. Create a Selenium Maven project.

2. Add Selenium and TestNG dependencies in pom.xml.
3. Configure ChromeDriver.
4. Create Selenium test cases.
5. Configure the Selenium project in Jenkins.
6. Set the Maven goal as clean test.
7. Execute the Jenkins build.
8. Verify the test execution report and console output.

Result

Thus, Selenium automation test cases were executed successfully using Jenkins and Maven.

Output



SCREENSHOT PLACEHOLDER(S)

- *Selenium Project*
 - *Jenkins Build*
 - *Test Report*
 - *BUILD SUCCESS*

Experiment No. 8

Docker Installation and Container Management

Aim

To install Docker and perform Docker image and container management operations.

Required Software

- Windows 10/11
- Docker Desktop
- Command Prompt or PowerShell
- Web Browser

Program / Commands

Docker Commands

```
docker --version
docker info
docker images
docker ps
docker ps -a
docker pull nginx
docker run -d -p 8080:80 nginx
docker stop <container_id>
docker start <container_id>
docker rm <container_id>
docker rmi nginx
```

Algorithm

1. Install Docker Desktop.
2. Verify the Docker installation.
3. Pull the Nginx image from Docker Hub.
4. Create and run a Docker container.
5. Display the running containers.
6. Stop and restart the container.
7. Remove the container and image.
8. Verify Docker images and containers.

Result

Thus, Docker was installed successfully, and Docker image and container management operations were performed successfully.

Output



SCREENSHOT PLACEHOLDER(S)

- *Docker Desktop*
- *Docker Version*
- *Running Container*
- *Docker Images*
- *Browser Output*

Experiment No. 9

Create Dockerfile and Build Docker Image for a Sample Web Application

Aim

To create a Dockerfile and build a Docker image for a sample web application.

Required Software

- Docker Desktop
- Visual Studio Code
- HTML Sample Project
- Command Prompt

Program / Commands

Dockerfile

```
FROM nginx:latest  
  
COPY . /usr/share/nginx/html  
  
EXPOSE 80
```

Build Docker Image

```
docker build -t sample-web .
```

Verify Docker Image

```
docker images
```

Run Docker Container

```
docker run -d -p 8080:80 sample-web
```

Algorithm

1. Create a sample web application.
2. Create a Dockerfile inside the project folder.

3. Specify the base image and copy application files.
4. Build the Docker image using the Dockerfile.
5. Verify that the Docker image is created successfully.
6. Run the Docker container using the created image.
7. Access the application through the browser.
8. Verify that the web application loads successfully.

Result

Thus, the Docker image was successfully created using Dockerfile, and the sample web application was executed successfully.

Output



SCREENSHOT PLACEHOLDER(S)

- *Dockerfile*
- *Docker Build*
- *Docker Images*
- *Running Container*
- *Browser Output*